

HeatDraft ML Pipeline: Project Defense Report

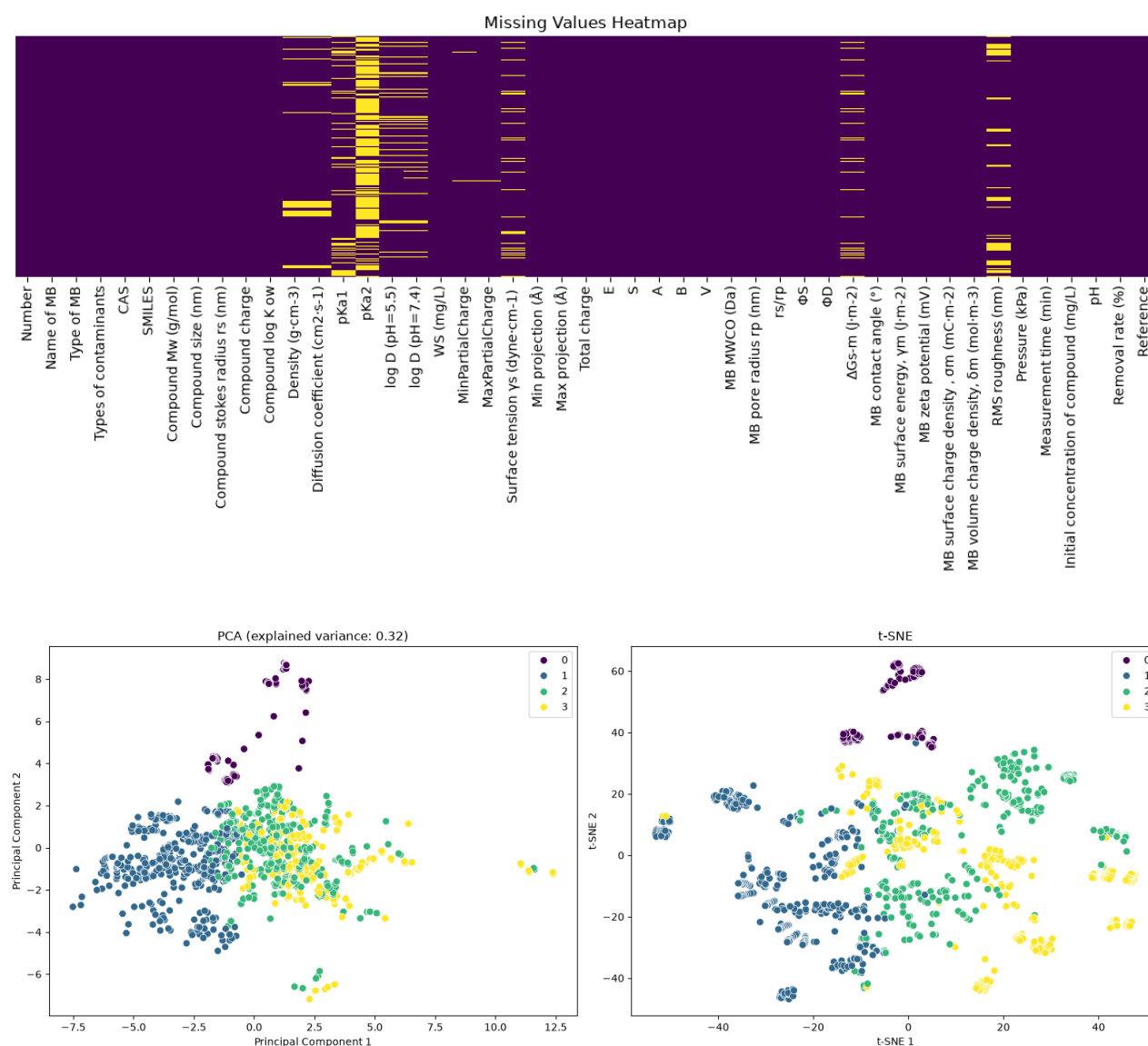
Part 1: Data Acquisition & EDA

What I did:

I built an automated module that generates statistical profiles, missing value heatmaps, and correlation matrices. Furthermore, I implemented Principal Component Analysis (PCA) and t-SNE to compress the chemical features into a 2D plane, grouping them using K-Means clustering (n=4).

Why I did it:

I needed to mathematically verify the skewness of the raw chemical properties before feeding them to the neural network. I implemented PCA and t-SNE to visually prove that distinct structural clusters of membrane-contaminant interactions actually exist in the data.



HeatDraft ML Pipeline: Project Defense Report

Part 2: Deterministic Chemical Imputation

What I did:

Instead of using standard data science techniques (like mean filling), I integrated RDKit to mathematically calculate intrinsic properties (MolWt, MolLogP) directly from SMILES strings. For properties that RDKit could not calculate locally, I built a dynamic web-scraper that connects to the NIH PubChem REST API to extract empirical pKa values.

Why I did it:

Standard statistical imputation is scientifically invalid for hard chemistry. A molecule's weight is a fixed physical constant, not an average of its neighbors in an Excel sheet. By falling back to the NIH PubChem database, I ensured the neural network was learning from true, peer-reviewed empirical laboratory values.

Part 3: The Tabular Attention Neural Network

What I did:

Instead of using a standard Multi-Layer Perceptron (MLP), I designed and implemented a custom TabularAttentionNet in PyTorch. I programmed it to project each scalar feature into a high-dimensional vector space, treating every spreadsheet column like a 'token'. I then passed these tokens through an nn.TransformerEncoderLayer featuring a Query-Key-Value (QKV) self-attention mechanism.

Why I did it:

Standard dense neural networks mash all columns together indiscriminately in the first layer. I chose a Transformer-based Attention mechanism because it allows the network to dynamically calculate 'Attention Weights' between different columns, simulating complex physics (like electrostatic repulsion) dynamically rather than relying on static weights.

HeatDraft ML Pipeline: Project Defense Report

Part 4: Final Evaluation and Post-Training Diagnostics

What I did:

I evaluated the final neural network weights on the completely unseen test set. I wrote a custom Permutation Feature Importance script to mathematically shuffle physics columns and measure the MSE explosion. I also built an Erratic Error Analysis visualization that plots Absolute Prediction Error against key physical constraints to identify boundary failures.